
Kimia KHADEMLOU, Abdel Kader EL HOUSSAMI

M1 MOSiG

PAP LAB 4 Report

CPU specifications:

Architecture:	x86_64
CPU op-mode(s):	32-bit, 64-bit
Byte Order:	Little Endian
Address sizes:	40 bits physical, 48 bits virtual
CPU(s):	8
On-line CPU(s) list:	0-7
Thread(s) per core:	1

Codes are tested on G5K.

EXERCISE 1. 1D blocking communication

Description of the work:

We implemented a 1D domain decomposition along the X axis. Each MPI process handles a part of the global grid.

We computed:

- the position of each process (rank_x)
- the local width and height
- the global position of the subdomain

Each process communicates with its left and right neighbors.

We used blocking MPI functions:

- MPI_Send
- MPI_Recv

We exchanged the border cells to update the ghost cells.

Communication

Each process sends:

- its left border to the left neighbor
- its right border to the right neighbor

Each process receives:

- data from the left neighbor
- data from the right neighbor

We handled boundary cases using MPI_PROC_NULL.

Results:

The communication works correctly. We verified using the following cmd:

mpirun -np 4 ./check_comm -e 1 -p position

EXERCISE 2. 1D odd/even communication

Description of the work:

We used the same 1D domain decomposition as in Exercise 1. Each process communicates with its left and right neighbors.

We still used blocking MPI functions:

- MPI_Send
- MPI_Recv

However, we changed the communication order using an odd/even scheme.

Communication

Processes are divided into two groups:

- even ranks
- odd ranks

Even ranks:

- receive first
- then send

Odd ranks:

- send first
- then receive

This avoids blocking situations and allows communications to progress in parallel.

Results:

The communication works correctly. We verified the communication using:

`mpirun -np 4 ./check_comm -e 2 -p position`

no errors were observed. We also compared the result with the sequential version using checksum. The checksum matched, which confirms that the result is correct.

IMPROVEMENT:

Compared to exo 1, This method reduces waiting between processes. Communication is more efficient than the basic blocking version.

EXERCISE 3. 1D non-blocking communication

Description of the work

We used the same 1D domain decomposition as in previous exercises.

Each process communicates with its left and right neighbors.

We implemented communication using non-blocking MPI functions:

- `MPI_Isend`
- `MPI_Irecv`
- `MPI_Waitall`

Communication

Each process posts all receive operations first, Then it sends its border data to neighbors.

After that, we call `MPI_Waitall` to ensure all communications are completed. This allows communications to overlap and avoids blocking between processes.

Result:

The communication works correctly.

We verified the communication using:

`mpirun -np 4 ./check_comm -e 3 -p position`

Improvement compared to Exercise 2 : This method allows communications to run in parallel. Processes do not need to wait before sending or receiving. This improves performance

EXERCISE 4. 2D blocking communication (manual copy)

Description of the work:

We implemented a 2D domain decomposition along the X and Y axes. Each process handles a subdomain of the global grid.

Each process communicates with up to 8 neighbors:

- left, right
- top, bottom
- top-left, top-right
- bottom-left, bottom-right

We used blocking MPI functions:

- MPI_Send
- MPI_Recv

Communication:

Left and right communications are contiguous in memory. Top and bottom communications are not contiguous. So we used temporary buffers to copy data before sending.

We copied:

- data from the border into send buffers
- received data into receive buffers
- then copied back into ghost cells

Results :

As for the results of ex 4, the tests don't really pass. The bug wasn't really found, but it is likely that the problem comes from the manipulation of the ghost cells, or the copying of the non-contiguous data.

EXERCISE 5. 2D blocking communication using MPI Datatypes

Description of the work:

We used the same 2D domain decomposition as in Exercise 4. Each process communicates with up to 8 neighbors.

We implemented communication using blocking MPI functions:

- MPI_Send
- MPI_Recv

We used MPI Datatypes to handle non-contiguous data.

Communication

Left and right communications are contiguous in memory, Top and bottom communications are not contiguous. Instead of using buffers, we defined a custom MPI datatype. This datatype allows sending columns directly from memory.

Result

The communication works correctly. We verified the communication using:

`mpirun -np 4 ./check_comm -e 5 -p position`

We also compared the result with the sequential version using checksum, and matched.

Improvement compared to Exercise 4 : This approach avoids manual data copies. The code is cleaner and easier to maintain. MPI handles non-contiguous memory directly.

EXERCISE 6. 2D non-blocking communication

Description of the work

We used the same 2D domain decomposition as in Exercise 5.

Each process communicates with up to 8 neighbors.

We implemented communication using non-blocking MPI functions:

- `MPI_Isend`
- `MPI_Irecv`
- `MPI_Waitall`

We also used MPI Datatypes to handle non-contiguous data.

Communication

Each process exchanges data with:

- left, right
- top, bottom
- and the 4 diagonal neighbors

We implemented the communication in two steps:

1. First, we exchange left, right, top and bottom borders.
2. Then, we exchange diagonal values.

For each step:

- we post all `MPI_Irecv` calls first
- then we call `MPI_Isend`
- finally we use `MPI_Waitall` to complete communication

This avoids blocking and allows parallel communication.

Implementation details

Following the tips, we used MPI Cartesian topology to get neighbor ranks. We also implemented a helper function to compute the rank from (x, y) coordinates. We used:

- MPI datatype for non-contiguous borders
- direct send for contiguous data

-
- MPI_PROC_NULL for boundary cases

Results:

The communication works correctly. We verified the communication using:

`mpirun -np 4 ./check_comm -e 6 -p position`

Checksum matches.

Conclusion

In this project, we implemented different MPI communication strategies for the Lattice Boltzmann simulation.

We started with simple 1D blocking communication. Then we improved it using odd/even and non-blocking methods.

After that, we extended the solution to 2D domain decomposition. We handled more complex communications with 8 neighbors. We also improved the implementation by using MPI Datatypes and non-blocking communication. All implementations were tested using the provided tools and gave correct results.

This project helped us understand:

- domain decomposition
- ghost cells
- MPI communication techniques